

Proyek I

Praktikum Pemrograman Dasar

Sistem Pengelolaan Peminjaman Peralatan Laboratorium

A. Deskripsi Proyek

Sebuah laboratorium di lingkungan universitas menyediakan berbagai peralatan yang dapat dipinjam oleh mahasiswa untuk kegiatan praktikum, penelitian, maupun proyek akademik.

Selama ini proses pencatatan peminjaman dilakukan secara manual sehingga sering terjadi beberapa permasalahan, seperti:

- status ketersediaan peralatan tidak tercatat dengan baik;
- mahasiswa dapat mengajukan peminjaman alat yang sebenarnya sedang digunakan;
- riwayat peminjaman sulit dilacak;
- keterlambatan pengembalian tidak terdokumentasi;
- kondisi alat setelah dikembalikan tidak tercatat;
- sulit mengetahui alat yang paling sering dipinjam atau mengalami kerusakan.

Laboratorium ingin mengembangkan sebuah **aplikasi berbasis Python** untuk mengelola proses tersebut.

Aplikasi dikembangkan menggunakan pendekatan **Object-Oriented Programming (OOP)** dengan memanfaatkan struktur data dinamis (built in collection) sebagai media penyimpanan data selama program berjalan.

B. Tujuan Proyek

Melalui proyek ini mahasiswa diharapkan mampu:

1. menganalisis permasalahan nyata dan menerjemahkannya menjadi rancangan program;
2. merancang struktur class dan hubungan antarclass menggunakan UML;
3. menerapkan konsep Object-Oriented Programming dalam Python;
4. menggunakan tipe data built-in collection di Python untuk mengelola data dinamis;
5. mengimplementasikan operasi CRUD;
6. mengembangkan program secara kolaboratif menggunakan GitHub;
7. mengevaluasi kualitas solusi melalui pengujian, presentasi, dan code review.

C. Kebutuhan Sistem

Sistem minimal harus mengelola beberapa jenis data berikut.

1. Data Mahasiswa

Setiap mahasiswa memiliki informasi:

- NIM;
- nama;
- nomor HP;
- status aktif peminjaman.

2. Data Peralatan

Setiap alat memiliki:

- kode alat;
- nama alat;
- kategori;
- kondisi alat.

Kategori alat dapat berbeda-beda, misalnya:

- perangkat komputasi;
- perangkat jaringan;
- perangkat multimedia;

Program harus memungkinkan penambahan kategori baru tanpa harus mengubah struktur utama program.

3. Transaksi Peminjaman

Satu mahasiswa dapat meminjam lebih dari satu alat dalam satu transaksi.

Setiap transaksi minimal menyimpan:

- ID transaksi;
- mahasiswa peminjam;
- daftar alat yang dipinjam;
- tanggal peminjaman;
- batas waktu pengembalian (maks 7 hari);
- status transaksi.

Contoh status transaksi:

- dipinjam;
- sebagian dikembalikan;
- selesai.

4. Pengembalian Alat

Ketika alat dikembalikan, sistem harus mencatat:

- alat yang dikembalikan;
- kondisi alat;
- tanggal pengembalian.

Kondisi alat setelah dikembalikan dapat berupa:

- baik;
- rusak ringan;
- rusak berat.

Peralatan yang rusak berat tidak boleh langsung dimasukkan kembali ke dalam stok alat yang tersedia (unavailable untuk dipinjam)

D. Aturan Bisnis

Program yang dikembangkan harus memenuhi aturan berikut.

Aturan 1: Ketersediaan Alat

Mahasiswa tidak dapat meminjam alat apabila tidak tersedia.

Aturan 2: Batas Peminjaman

Seorang mahasiswa hanya diperbolehkan memiliki maksimal **dua transaksi peminjaman aktif**.

Aturan 3: Isi Transaksi

Satu transaksi dapat berisi beberapa jenis alat.

Contoh:

Mahasiswa A meminjam:

- kamera digital;
- tripod;
- kabel LAN.

Informasi tersebut tidak boleh dibuat menggunakan variabel terpisah untuk setiap alat.

Gunakan struktur data yang memungkinkan jumlah dan jenis alat berkembang secara dinamis.

Aturan 4: Pengembalian Sebagian

Mahasiswa diperbolehkan mengembalikan hanya sebagian alat terlebih dahulu.

Transaksi tetap dianggap aktif sampai seluruh alat dikembalikan.

Aturan 5: Kondisi Alat

Jumlah alat tersedia harus disesuaikan dengan kondisi alat saat dikembalikan.

Alat dengan kondisi **baik** dapat langsung kembali menjadi stok tersedia.

Alat dengan kondisi **rusak ringan** atau **rusak berat** tidak boleh dihitung sebagai alat tersedia.

Aturan 6: Penghapusan Data

Data mahasiswa tidak boleh dihapus apabila mahasiswa tersebut masih memiliki transaksi peminjaman aktif.

Data alat juga tidak boleh dihapus apabila alat tersebut masih tercatat dalam transaksi peminjaman aktif.

E. Fitur Minimal Program

Program harus menyediakan menu interaktif sekurang-kurangnya sebagai berikut:

1. Kelola data mahasiswa (tambah, edit, hapus, cari)
2. Kelola data alat (tambah, edit, hapus, cari)
3. Buat transaksi peminjaman
4. Tampilkan transaksi
5. Proses pengembalian alat
6. Cari transaksi berdasarkan mahasiswa
7. Tampilkan alat yang tersedia
8. Tampilkan alat yang sedang dipinjam
9. Tampilkan alat yang rusak
10. Tampilkan riwayat peminjaman mahasiswa
11. Keluar dari program

Menu boleh dikembangkan sesuai rancangan masing-masing kelompok.

F. Ketentuan Teknis

Program harus memenuhi ketentuan berikut:

- menggunakan Python;
- menggunakan paradigma Object-Oriented Programming;
- memiliki lebih dari satu class;
- menggunakan List dan/atau Dictionary sebagai struktur data utama;

Mahasiswa **tidak diperbolehkan menggunakan database** seperti MySQL, SQLite, PostgreSQL, MongoDB, atau sejenisnya.

Tahap Penyelesaian Proyek

Tahap 1: Perancangan Solusi

Tugas Individu

Sebelum melakukan diskusi kelompok, **setiap anggota kelompok wajib membuat rancangan solusi secara mandiri.**

Setiap mahasiswa harus membuat **UML Class Diagram menggunakan Draw.io.**

Rancangan minimal menunjukkan:

- class yang diperlukan;
- atribut setiap class;
- method utama;
- hubungan antarclass;
- struktur tanggung jawab masing-masing class.

Tidak ditentukan berapa jumlah class yang harus dibuat.

Mahasiswa harus menentukan sendiri struktur class yang dianggap paling tepat berdasarkan analisis terhadap permasalahan.

Setiap mahasiswa harus dapat menjelaskan:

Mengapa struktur class tersebut dipilih dan bagaimana masing-masing class bekerja sama untuk menyelesaikan permasalahan?

Diskusi Kelompok

Setelah seluruh anggota menyelesaikan rancangan masing-masing, lakukan diskusi kelompok.

Setiap anggota harus menjelaskan/menyajikan rancangan miliknya.

Kelompok kemudian:

1. membandingkan setiap rancangan;
2. mengidentifikasi kelebihan dan kekurangannya;
3. menentukan rancangan terbaik;
4. melakukan perbaikan apabila diperlukan;
5. menghasilkan **satu UML Class Diagram final kelompok**.

Rancangan akhir **tidak harus sama persis dengan salah satu rancangan individu**.

Kelompok diperbolehkan menggabungkan bagian terbaik dari beberapa rancangan.

Dokumen Keputusan Desain

Kelompok harus menyertakan penjelasan singkat mengenai:

- rancangan mana yang menjadi dasar solusi kelompok;
- bagian mana yang diambil atau dimodifikasi dari rancangan anggota lain;
- alasan memilih struktur class tersebut;
- alternatif desain yang sempat dipertimbangkan tetapi tidak dipilih.

Tahap 2: Implementasi Solusi

Rancangan kelompok selanjutnya diimplementasikan menjadi program Python.

Pengembangan dilakukan secara kolaboratif menggunakan **GitHub**.

Setiap anggota kelompok harus memiliki kontribusi terhadap repository.

Kontribusi dapat berupa:

- membuat class tertentu;
- mengembangkan fitur tertentu;
- melakukan validasi input;
- melakukan testing;
- memperbaiki bug;
- melakukan refactoring;
- memperbaiki dokumentasi program.

Setiap anggota harus menggunakan akun GitHub masing-masing.

Kelompok tidak diperbolehkan mengerjakan seluruh program menggunakan satu akun GitHub saja.

Ketentuan Repository

Repository minimal memiliki:

```
None
README.md
main.py
folder/module program
folder dokumentasi
```

README harus menjelaskan:

- nama aplikasi;
- anggota kelompok;
- deskripsi sistem;
- struktur program;
- cara menjalankan aplikasi;
- pembagian kontribusi anggota.

Tantangan Pengembangan

Setelah seluruh kebutuhan dasar berhasil dibuat, pilih minimal **dua** dari tantangan berikut.

NB: Tantangan ini bersifat penambahan fitur pada aplikasi yang dikembangkan.

Tantangan A: Pencarian Fleksibel

Pengguna dapat mencari alat berdasarkan:

- kode;
- sebagian nama alat;
- kategori.

Tantangan B: Statistik Peminjaman

Program dapat menampilkan:

- alat yang paling sering dipinjam;

- mahasiswa yang paling sering melakukan peminjaman;
- jumlah transaksi yang telah selesai;
- jumlah transaksi yang masih aktif.

Tantangan C: Pemeliharaan Peralatan

Alat yang rusak berat dapat dimasukkan ke daftar pemeliharaan.

Setelah selesai diperbaiki, alat tersebut dapat dikembalikan ke stok tersedia.

Tantangan D: Log Activities

Program menyimpan riwayat/log aktivitas, misalnya:

Python

```
[2026-08-10 10:15] Mahasiswa M001 ditambahkan
[2026-08-10 10:20] Transaksi T001 dibuat
[2026-08-10 10:30] Dua unit Multimeter dipinjam
[2026-08-10 13:45] Satu unit Multimeter dikembalikan
```

Keterangan: Data log activities juga disimpan dalam list atau dictionary.

Tahap 3: Evaluasi Solusi

Setelah program selesai dikembangkan, setiap kelompok melakukan presentasi.

Presentasi minimal menunjukkan:

1. permasalahan yang diselesaikan;
2. UML final;
3. alasan pemilihan struktur class;
4. demonstrasi program;
5. struktur List dan Dictionary yang digunakan;
6. pembagian tugas anggota;
7. histori pengembangan melalui GitHub;
8. kendala teknis selama pengembangan;
9. perubahan desain yang terjadi dari UML awal hingga program akhir.

Cross-Group Code Review

Setiap kelompok akan melakukan review terhadap program kelompok lain.

Reviewer tidak hanya mencoba menjalankan program, tetapi juga membaca struktur kode.

Review dilakukan berdasarkan aspek berikut.

1. Kesesuaian Rancangan dan Implementasi

Periksa apakah:

- class pada UML benar-benar diimplementasikan;
- atribut sesuai rancangan;
- method sesuai rancangan;
- hubungan antarclass sesuai dengan implementasi;

2. Ketepatan Solusi

Periksa apakah:

- program dapat dijalankan;
- operasi CRUD berfungsi;
- transaksi peminjaman berjalan dengan benar;
- status ketersediaan alat diperbarui dengan benar;
- pengembalian sebagian dapat diproses;
- aturan bisnis diterapkan;

3. Skalabilitas Program

Bayangkan bahwa sistem berkembang dari:

None

50 alat

100 mahasiswa

menjadi:

None

10.000 alat

20.000 mahasiswa

atau memiliki kebutuhan baru seperti:

- kategori alat baru;
- jenis pengguna baru;
- aturan peminjaman baru;

- ada denda keterlambatan
- penyimpanan menggunakan database;

Reviewer harus mengevaluasi:

Apakah struktur program saat ini cukup mudah dikembangkan untuk kebutuhan tersebut?

Identifikasi bagian kode yang kemungkinan menjadi sulit dipelihara apabila sistem berkembang.

Skenario Pengujian Wajib

Setiap kelompok harus menguji programnya minimal menggunakan skenario berikut.

Skenario 1

Tambahkan beberapa mahasiswa dan beberapa jenis peralatan.

Skenario 2

Mahasiswa meminjam beberapa alat dalam satu transaksi.

Periksa perubahan status alat.

Skenario 3

Mahasiswa mencoba meminjam alat yang tidak tersedia.

Program harus menolak transaksi.

Skenario 4

Mahasiswa mengembalikan sebagian alat.

Periksa:

- status ketersediaan alat;
- status transaksi;
- data alat yang masih dipinjam.

Skenario 5

Salah satu alat dikembalikan dalam kondisi rusak berat atau rusak ringan.

Periksa apakah alat tersebut kembali tersedia atau tidak.

Skenario 6

Mahasiswa yang masih memiliki transaksi aktif mencoba dihapus.

Program harus menolak operasi tersebut.

Pertanyaan Refleksi Kelompok

Setelah code review, setiap kelompok menjawab pertanyaan berikut:

1. Apa kelemahan utama rancangan solusi kelompok kalian?
2. Jika aplikasi ini akan dikembangkan menjadi sistem nyata, bagian apa yang pertama kali perlu diperbaiki?
3. Apakah terdapat class yang memiliki terlalu banyak tanggung jawab?
4. Apakah struktur data collection yang digunakan sudah tepat?
5. Apakah terdapat duplikasi kode (ada bagian kode yang ditulis bbrp kali)?
6. Bagaimana desain struktur class dapat diperbaiki agar lebih scalable?
7. Perubahan apa yang kalian lakukan setelah menerima hasil code review dari kelompok lain?

Artefak yang Dikumpulkan

Setiap kelompok mengumpulkan:

Artefak Individu

- UML Class Diagram setiap anggota.

Artefak Kelompok

- UML Class Diagram final;
- dokumen keputusan desain (disertai alasan mengapa dipilih desain tersebut);
- URL repository GitHub;
- source code program;
- README;
- dokumen hasil pengujian;
- dokumen hasil cross-group code review;
- dokumen refleksi dan rencana perbaikan solusi.